

Тема работы: Оконный интерфейс и оконные приложения, обработка оконных сообщений, базовая графика GDI

Цель работы: Возобновление, закрепление и развитие навыков программирования оконных приложений Windows: структура приложения, цикл обработки сообщений, организация взаимодействия посредством сообщений, создание и использование окон и оконных элементов управления, использование базовых средств графики Windows.

Выполнение работы

Демонстрационное оконное приложение с интерфейсом пользователя (GUI) «базового» уровня: имеющее область отображения графических объектов, снабженное элементами управления и реагирующее на ограниченный набор событий (сообщений). (Возможны модификации заданий, в т.ч. по инициативе учащихся. Также по желанию и/или взаимной договоренности может повышаться уровень сложности.) Поскольку тема – работа с базовыми возможностями системы, программа должна демонстрировать умение пользоваться соответствующими системными вызовами, т.е. Win API. Вероятно, языки C/C++ подходят для этого в наибольшей степени, однако возможны и иные варианты.

8 Представление данных диаграммой Оконное приложение, отображающее содержимое заданного файла диаграммой (или графиком): файл содержит последовательность чисел, которые считаются одномерным массивом, каждое значение соответствует интервалу диаграммы (или точке графика). Размерность таблицы можно считать фиксированной или изменять в соответствии с актуальным набором данных. Выбор файла через стандартный диалог или передача имени в командной строке.

Листинг кода программы:

```
#include <windows.h>
#include <commdlg.h>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <cctype>

std::vector<double> g_data;
std::wstring g_filename;

LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
void LoadDataFromFile(const std::wstring& filename);
void DrawChart(HDC hdc, const RECT& rc);

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine,
int nCmdShow) {
    int argc;
    LPWSTR cmdLine = GetCommandLineW();
    LPWSTR* argv = CommandLineToArgvW(cmdLine, &argc);
```

```

    if (argc > 1) {
        g_filename = argv[1];
        LoadDataFromFile(g_filename);
    }

    LocalFree(argv);

    const wchar_t CLASS_NAME[] = L"ChartWindowClass";

    WNDCLASS wc = {};
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = CLASS_NAME;
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);

    RegisterClass(&wc);

    HWND hwnd = CreateWindowEx(
        0,
        CLASS_NAME,
        L"Диаграмма данных",
        WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, CW_USEDEFAULT, 800, 600,
        NULL,
        NULL,
        hInstance,
        NULL
    );

    if (hwnd == NULL) {
        return 0;
    }

    ShowWindow(hwnd, nCmdShow);

    MSG msg = {};
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return (int)msg.wParam;
}

// Обработчик сообщений окна
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
        case WM_CREATE: {
            if (g_data.empty()) {
                OPENFILENAME ofn = {};
                wchar_t szFile[260] = { 0 };

                ofn.lStructSize = sizeof(ofn);
                ofn.hwndOwner = hwnd;
                ofn.lpstrFile = szFile;
                ofn.nMaxFile = sizeof(szFile);
                ofn.lpstrFilter = L"Текстовые файлы (*.txt)\0*.txt\0Все файлы (*.*)\0*.*\0";
                ofn.nFilterIndex = 1;
                ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST;

                if (GetOpenFileName(&ofn)) {
                    g_filename = ofn.lpstrFile;
                    LoadDataFromFile(g_filename);
                }
            }
        }
    }
}

```

```

        InvalidateRect(hwnd, NULL, TRUE);
    }
    else {
        PostQuitMessage(0);
    }
}
break;
}

case WM_PAINT: {
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hwnd, &ps);
    RECT rc;
    GetClientRect(hwnd, &rc);
    DrawChart(hdc, rc);
    EndPaint(hwnd, &ps);
    break;
}

case WM_SIZE:
    InvalidateRect(hwnd, NULL, TRUE);
    break;

case WM_DESTROY:
    PostQuitMessage(0);
    break;

default:
    return DefWindowProc(hwnd, msg, wParam, lParam);
}
return 0;
}

// Загрузка данных из файла
void LoadDataFromFile(const std::wstring& filename) {
    g_data.clear();

    std::ifstream file(filename);
    if (!file.is_open()) return;

    std::string line;
    while (std::getline(file, line)) {
        line.erase(line.begin(), std::find_if(line.begin(), line.end(), [](unsigned
char ch) {
            return !std::isspace(ch);
        }));
        line.erase(std::find_if(line.rbegin(), line.rend(), [](unsigned char ch) {
            return !std::isspace(ch);
        }).base(), line.end());

        if (line.empty()) continue;

        try {
            double val = std::stod(line);
            g_data.push_back(val);
        }
        catch (...) {}
    }
    file.close();
}

// Отрисовка диаграммы
void DrawChart(HDC hdc, const RECT& rc) {
    if (g_data.empty()) {

```

```

        DrawText(hdc, L"Нет данных", -1, const_cast<RECT*>(&rc), DT_CENTER |
DT_VCENTER | DT_SINGLELINE);
        return;
    }

    int width = rc.right - rc.left;
    int height = rc.bottom - rc.top;

    double minVal = *std::min_element(g_data.begin(), g_data.end());
    double maxVal = *std::max_element(g_data.begin(), g_data.end());
    double range = maxVal - minVal;
    if (range == 0.0) range = 1.0;

    int margin = 20;
    int chartWidth = width - 2 * margin;
    int chartHeight = height - 2 * margin;

    double scaleX = (double)chartWidth / (double)g_data.size();
    double scaleY = (double)chartHeight / range;

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(180, 180, 255));
    HBRUSH hBrush = CreateSolidBrush(RGB(200, 50, 50));
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);
    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

    for (size_t i = 0; i < g_data.size(); ++i) {
        double x = margin + i * scaleX;
        double value = g_data[i];
        double barHeight = (value - minVal) * scaleY;

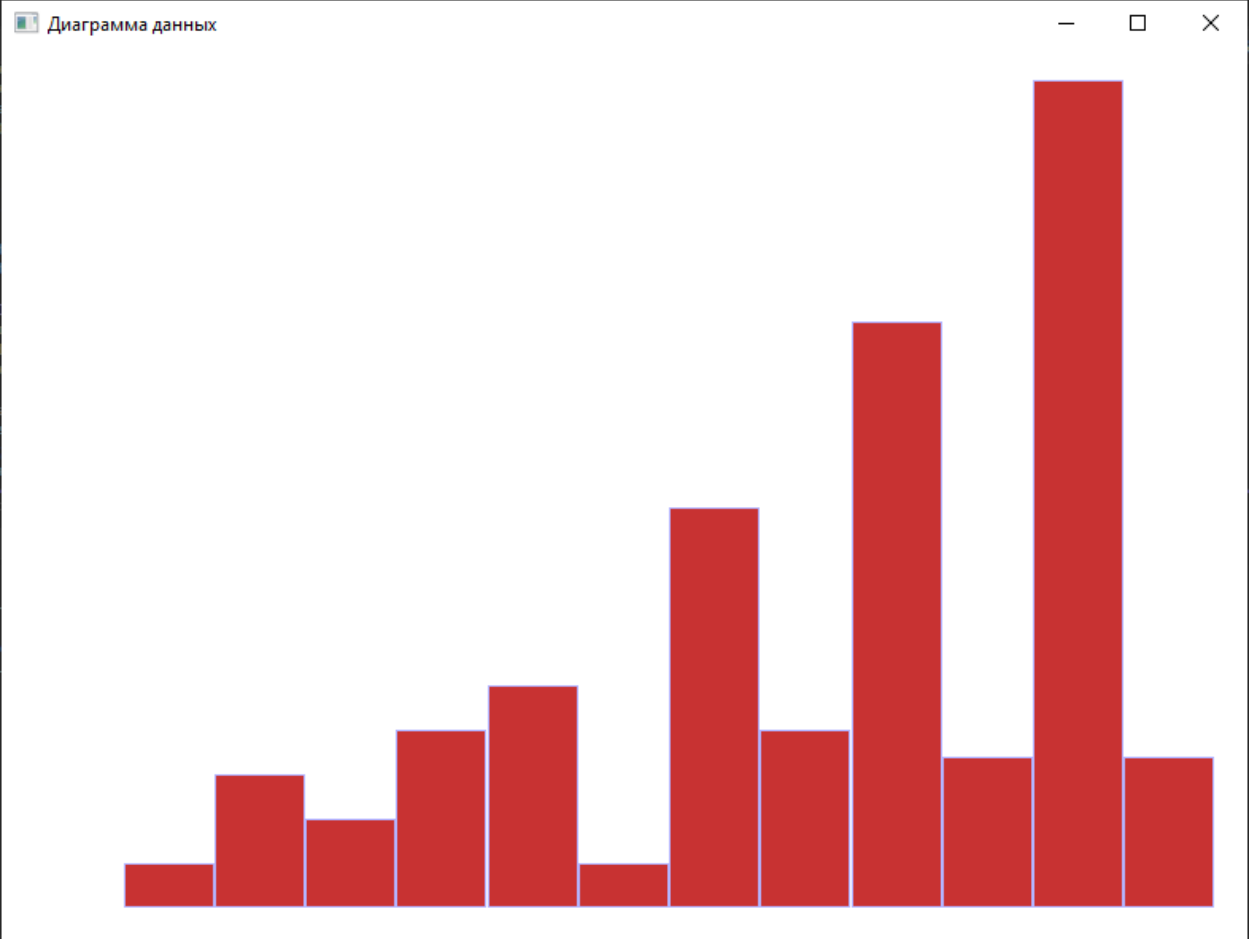
        int top = rc.bottom - margin - (int)barHeight;
        int bottom = rc.bottom - margin;
        int barWidth = (int)(scaleX > 1 ? scaleX : 1);

        Rectangle(hdc, (int)x, top, (int)(x + barWidth), bottom);
    }

    SelectObject(hdc, hOldPen);
    SelectObject(hdc, hOldBrush);
    DeleteObject(hPen);
    DeleteObject(hBrush);
}

```

Результат работы программы:



Выводы

В данной лабораторной работе я изучил программирование оконных приложений Windows: структуру приложения, цикл обработки сообщений, организацию взаимодействия посредством сообщений, создание и использование окон и оконных элементов управления, использование базовых средств графики Windows.